



(SQL) Displaying Data from Multiple Tables

Talia Noureen

Beaconhouse National University
School of Computer and Information Technology (SCIT)

Objectives

After completing this lesson, you should be able to do the following:

- Write `SELECT` statements to access data from more than one table using equality and non-equality joins
- View data that generally does not meet a join condition by using outer joins
- Join a table to itself by using a self join

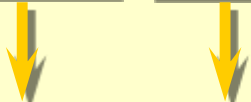
Obtaining Data from Multiple Tables

EMP

EMPNO	ENAME	...	DEPTNO
-----	-----	...	-----
7839	KING	...	10
7698	BLAKE	...	30
...			
7934	MILLER	...	10

DEPT

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON



EMPNO	DEPTNO	LOC
-----	-----	-----
7839	10	NEW YORK
7698	30	CHICAGO
7782	10	NEW YORK
7566	20	DALLAS
7654	30	CHICAGO
7499	30	CHICAGO
...		
14 rows selected.		

What Is a Join?

- Use a join to query data from more than one table.

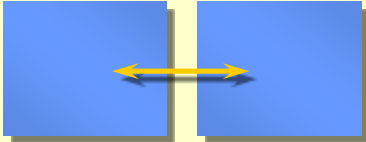
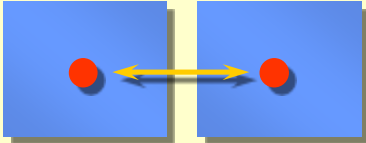
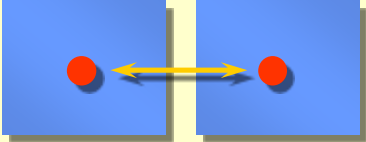
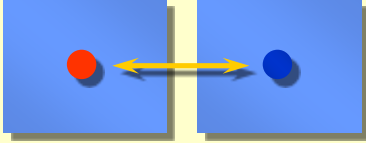
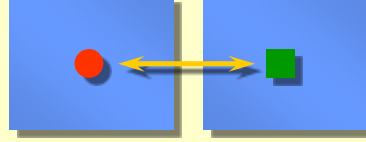
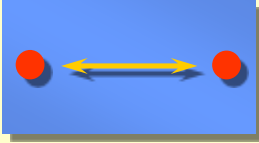
```
SELECT    table1.column, table2.column
FROM      table1, table2
WHERE     table1.column1 = table2.column2;
```

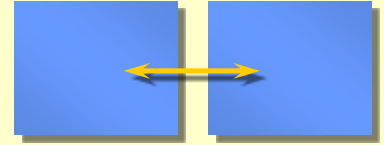
- Write the join condition in the WHERE clause.
- Prefix the column name with the table name when the same column name appears in more than one table.
- An alternate syntax for join operation introduced with **oracle 9i (ANSI syntax)**:

```
SELECT    table1.column, table2.column
FROM      table1 {join type} JOIN table2
[ON       table1.column1 = table2.column2];
```

- Join type can be INNER (EQUI), LEFT, RIGHT, FULL OUTER...

Types of Joins

Cartesian Join	
Natural Join	
Equi or Inner Join	
Outer Join	
Non-Equi Join	
Self Join	



1. Cross Join or Cartesian Product

- A Cartesian product is formed when:
 - A join condition is omitted
 - A join condition is invalid
- `Select ename, dname from emp,dept;` OR
- `Select ename, dname from emp cross join dept;`
- All rows in the first table are joined to all rows in the second table
- The number of rows returned by cross join is the product of the number of rows in each table.
- To avoid a Cartesian product, always include a valid join condition in a WHERE clause.

Generating a Cartesian Product

EMP (14 rows)

EMPNO	ENAME	...	DEPTNO
-----	-----	...	-----
7839	KING	...	10
7698	BLAKE	...	30
...			
7934	MILLER	...	10

DEPT (4 rows)

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

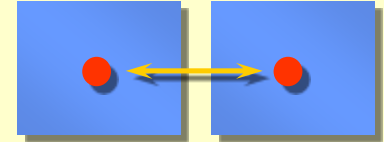


**“Cartesian
product:
14*4=56 rows”**



ENAME	DNAME
-----	-----
KING	ACCOUNTING
BLAKE	ACCOUNTING
...	
KING	RESEARCH
BLAKE	RESEARCH
...	
56 rows selected.	

Joining Tables with Common Columns



EMP

EMPNO	ENAME	DEPTNO
7839	KING	10
7698	BLAKE	30
7782	CLARK	10
7566	JONES	20
7654	MARTIN	30
7499	ALLEN	30
7844	TURNER	30
7900	JAMES	30
7521	WARD	30
7902	FORD	20
7369	SMITH	20
...		
14 rows selected.		

DEPT

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
30	SALES	CHICAGO
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
30	SALES	CHICAGO
30	SALES	CHICAGO
30	SALES	CHICAGO
30	SALES	CHICAGO
20	RESEARCH	DALLAS
20	RESEARCH	DALLAS
...		
14 rows selected.		

Foreign key Primary key

2. Natural Join

- The join is based on all columns that have same name in both tables.
- The join condition **should NOT be specified**.

```
SELECT col-1, col-2, ....  
FROM table-1 NATURAL [INNER] JOIN table-2  
[WHERE clause]
```

```
SELECT ename,dname  
FROM emp NATURAL JOIN dept
```

```
SELECT ename,dname  
FROM emp NATURAL INNER JOIN dept
```

3. Euijoin / Inner Join

- Inner joins are the most commonly used joins. For this reason, the term **join defaults to "inner join"**.
- The **pre-requisite** for inner join is that its participating tables must have **at least one common attribute** between them.

```
SELECT col-1, col-2, ....  
FROM table-1, table-2, ...  
WHERE table-1.Join_Col = table-2. Join_Col ...
```

```
SELECT col-1, col-2, ....  
FROM table-1 [INNER] JOIN table-2, ...  
{ON table-1. Join_Col = table-2. Join_Col ... | USING (Join_Col)}  
[WHERE clause]
```

3. Euijoin / Inner Join

- Select ename, dname from emp,dept
where emp.deptno = dept.deptno;
- Select ename, dptno, dname
from emp INNER JOIN dept
ON emp.deptno = dept.deptno;
- Select ename, dname from emp JOIN dept
ON emp.deptno = dept.deptno;
- Select ename, dname from emp JOIN dept
USING (deptno);
- Select e.empno, e.ename, d.deptno, d.dname
from emp e JOIN dept d
ON e.deptno = d.deptno;

Retrieving Records with Equijoin

```
SELECT    emp.empno, emp.ename, emp.deptno,
          dept.deptno, dept.loc
FROM      emp, dept
WHERE     emp.deptno = dept.deptno;
```

EMPNO	ENAME	DEPTNO	DEPTNO	LOC
7839	KING	10	10	NEW YORK
7698	BLAKE	30	30	CHICAGO
7782	CLARK	10	10	NEW YORK
7566	JONES	20	20	DALLAS
...				

14 rows selected.

Ambiguous Column Names

```
SELECT  empno, deptno, dname
        FROM      emp, dept
WHERE   emp.deptno = dept.deptno;
```

same column name in two tables

```
SELECT empno, deptno, dname
      *
ERROR at line 1:
ORA-00918: column ambiguously define
```

Qualifying Ambiguous Column Names

- Use table prefixes to qualify column names that are in multiple tables.

```
SELECT    empno, emp.deptno, dname
          FROM      emp, dept
WHERE     emp.deptno = dept.deptno;
```

Ambiguous Column Names – Natural JOIN

```
SELECT      empno, emp.deptno, dname
FROM        emp NATURAL JOIN dept
```

```
SELECT empno, emp.deptno, dname
      *
ERROR at line 1:
ORA-25155: column used in NATURAL JOIN
cannot have qualifier
```

```
SELECT      empno, deptno, dname
FROM        emp NATURAL JOIN dept
```

```
SELECT      e.empno, deptno, d.dname
FROM        emp e NATURAL JOIN dept d
```

Using Table Aliases

- Simplify queries by using table aliases.

```
SQL> SELECT emp.empno, emp.ename, emp.deptno,  
2          dept.deptno, dept.loc  
3 FROM    emp, dept  
4 WHERE   emp.deptno=dept.deptno;
```

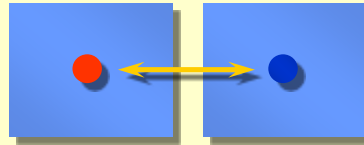
```
SQL> SELECT e.empno, e.ename, e.deptno,  
2          d.deptno, d.loc  
3 FROM    emp e, dept d  
4 WHERE   e.deptno=d.deptno;
```


Additional Search Conditions

Using the AND Operator

```
SELECT  emp.empno, emp.ename, emp.deptno,  
        dept.deptno, dept.loc  
FROM    emp, dept  
WHERE   emp.deptno=dept.deptno  
        AND dept.dname like 'SALES';
```

3. Outer Joins

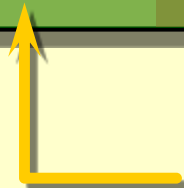


EMP

ENAME	DEPTNO
-----	-----
KING	10
BLAKE	30
CLARK	10
JONES	20
...	

DEPT

DEPTNO	DNAME
-----	-----
10	ACCOUNTING
30	SALES
10	ACCOUNTING
20	RESEARCH
...	
40	OPERATIONS



**No employee in the
OPERATIONS department**

Outer Joins

- Sometimes, while performing a join between two tables, you need to return all the rows from one table even when there are no corresponding rows in the other table.
- The inner join returns only those rows from the two tables that satisfy the join condition. Thus equijoin will not return the departments that don't have any employee or employees whose department is NULL.

FROM *table1* { LEFT | RIGHT | FULL } [OUTER]

JOIN *table2*

ON *table-1. Join_Col* = *table-2. Join_Col*

Outer Joins

- **LEFT**

Specifies that the results be generated using all rows from *table1*. For those rows in *table1* that don't have corresponding rows in *table2*, NULLs are returned in the result set for the *table2* columns.

- **RIGHT**

Specifies that the results be generated using all rows from *table2*. For those rows in *table2* that don't have corresponding rows in *table1*, NULLs are returned in the result set for the *table1* columns.

- **FULL**

Specifies that the results be generated using all rows from *table1* and *table2*. For those rows in *table1* that don't have corresponding rows in *table2*, NULLs are returned in the result set for the *table2* columns and vice-versa.

- **OUTER**

Specifies that you are performing an OUTER join. This keyword is optional. If you use LEFT, RIGHT, or FULL, Oracle automatically assumes an outer join. The OUTER keyword is for completeness' sake, and complements the INNER keyword.

Outer Joins

- You use an outer join to also see rows that do not usually meet the join condition.
- Outer join operator is the plus sign (+).
- Left Outer Join
 - Column of table1 has a NULL values

```
SELECT table1.column, table2.column  
FROM   table1, table2  
WHERE  table1.column = table2.column (+);
```

– Right Outer Join

- Column of table2 has a NULL values

```
SELECT table1.column, table2.column  
FROM   table1, table2  
WHERE  table1.column (+) = table2.column ;
```

Left Outer Joins

To list any employees not assigned to a particular department.

```
SQL> SELECT    e.ename, d.deptno, d.dname
  2  FROM      emp e, dept d
  3  WHERE     e.deptno = d.deptno(+)
```

```
SQL> SELECT    e.ename, d.deptno, d.dname
  2  FROM      emp e LEFT JOIN dept d
  3  ON        e.deptno = d.deptno
```

ENAME	DEPTNO	DNAME
-----	-----	-----
KING	10	ACCOUNTING
CLARK	10	ACCOUNTING
...		
BABU KHAN		

15 rows selected.

Right Outer Joins

To retrieve the departments that don't have any employees.

```
SQL> SELECT    e.ename, d.deptno, d.dname
  2  FROM      emp e, dept d
  3  WHERE     e.deptno(+) = d.deptno
```

```
SQL> SELECT    e.ename, d.deptno, d.dname
  2  FROM      emp e RIGHT OUTER JOIN dept d
  3  ON        e.deptno = d.deptno
```

ENAME	DEPTNO	DNAME
-----	-----	-----
KING	10	ACCOUNTING
CLARK	10	ACCOUNTING
...		
	40	OPERATIONS

15 rows selected.

Illegal Queries

Using Outer Joins

```
SQL> SELECT    e.ename, d.deptno, d.dname
 2  FROM        emp e, dept d
 3  WHERE       e.deptno(+) = d.deptno(+)
 4  ORDER BY   e.deptno;
```

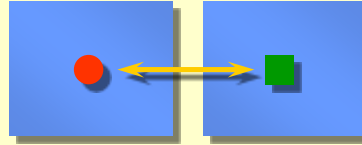
*Cannot use left and right outer
join simultaneously.*

Using Left and Right Outer Join in One Query

```
SQL> SELECT      e.ename, d.deptno, d.dname
  2  FROM          emp e, dept d
  3  WHERE         e.deptno(+) = d.deptno
  4  UNION
  5  SELECT      e.ename, d.deptno, d.dname
  6  FROM          emp e, dept d
  3  WHERE         e.deptno = d.deptno(+)
```

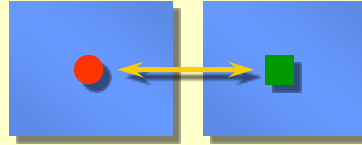
- Similarly, We can use **INTERSECT** and **MINUS** to combine two or more select statements into a single query, provided that the select-list of all the queries remain the same.

```
SQL> SELECT      e.ename, d.deptno, d.dname
  2  FROM          emp e FULL OUTER JOIN dept d
  3  ON            e.deptno = d.deptno
```



4. Non-Equijoins

- The join condition determines whether a join is an equi-join or a non-equijoin.
- When a join condition relates two tables by equating the columns from the tables, it is an equijoin.
- When a join condition relates two tables by an operator other than equality, it is a non-equijoin.
- Non-equijoin is used to retrieve data from two tables that don't have a common attribute between them, but still the tables are related with each other.



4. Non-Equijoins

EMP

EMPNO	ENAME	SAL
7839	KING	5000
7698	BLAKE	2850
7782	CLARK	2450
7566	JONES	2975
7654	MARTIN	1250
7499	ALLEN	1600
7844	TURNER	1500
7900	JAMES	950
...		
14 rows selected.		

SALGRADE

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

“salary in the EMP table is between low salary and high salary in the SALGRADE table”

Retrieving Records with Non-Equijoins

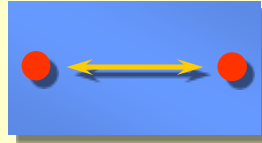
```
SQL>  SELECT    e.ename, e.sal, s.grade
      2  FROM      emp e, salgrade s
      3  WHERE     e.sal
      4  BETWEEN  s.losal AND s.hisal;
```

```
SQL>  SELECT    e.ename, e.sal, s.grade
      2  FROM      emp e JOIN salgrade s
      3  ON        e.sal
      4  BETWEEN  s.losal AND s.hisal;
```

ENAME	SAL	GRADE
-----	-----	-----
JAMES	950	1
SMITH	800	1
ADAMS	1100	1
...		

14 rows selected.

5. Self Joins

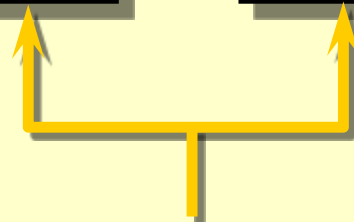


EMP (WORKER)

EMPNO	ENAME	MGR
-----	-----	-----
7839	KING	
7698	BLAKE	7839
7782	CLARK	7839
7566	JONES	7839
7654	MARTIN	7698
7499	ALLEN	7698

EMP (MANAGER)

EMPNO	ENAME
-----	-----
7839	KING
7839	KING
7839	KING
7698	BLAKE
7698	BLAKE



“MGR in the WORKER table is equal to EMPNO in the MANAGER table”

One row in a table is related to another row of the same table.

Joining a Table to Itself

```
SELECT e.ename "Employee", m.ename "Manager"
FROM   emp e, emp m
WHERE  e.mgr = m.empno;
```

```
SELECT e.ename "Employee", m.ename "Manager"
FROM   emp e JOIN emp m
ON     e.mgr = m.empno;
```

Employee	Manager

BLAKE	KING
CLARK	KING
JONES	KING
MARTIN	BLAKE
...	

13 rows selected.

Joining a Table to Itself

```
SELECT worker.ename||' works for '||manager.ename  
FROM   emp worker, emp manager  
WHERE  worker.mgr = manager.empno;
```

```
SELECT worker.ename||' works for '||manager.ename  
FROM   emp worker JOIN emp manager  
ON     worker.mgr = manager.empno;
```

```
WORKER.ENAME||'WORKSFOR'||MANAG
```

```
-----
```

```
BLAKE works for KING  
CLARK works for KING  
JONES works for KING  
MARTIN works for BLAKE  
...  
13 rows selected.
```

Self Outer Join

The last query doesn't return the employee, whose mgr is NULL. To include manager less employees, the following query is used:

```
SELECT e.ename "Employee", m.ename "Manager"
FROM   emp e LEFT OUTER JOIN emp m
ON     e.mgr = m.empno;
```

Employee	Manager

BLAKE	KING
CLARK	KING
JONES	KING
MARTIN	BLAKE
KING	
...	
14 rows selected.	

Joining More Than Two Tables

CUSTOMER

NAME	CUSTID
-----	-----
JOCKSPORTS	100
TKB SPORT SHOP	101
VOLLYRITE	102
JUST TENNIS	103
K+T SPORTS	105
SHAPE UP	106
WOMENS SPORTS	107
...	...
9 rows selected.	

ORD

CUSTID	ORDID
-----	-----
101	610
102	611
104	612
106	601
102	602
106	
106	
...	...
21 rows selected.	

ITEM

ORDID	ITEMID
-----	-----
610	3
611	1
612	1
601	1
602	1
...	
64 rows selected.	

- To join n tables together, you need a minimum of $n-1$ join conditions. For example, to join three tables, a minimum of two joins is required.

Joining More Than Two Tables

```
SELECT col-1, col-2, ....  
FROM table-1, table-2, ..., table-n  
WHERE table-1.Join_Col = table-2. Join_Col AND  
       table-1.Join_Col = table-3. Join_Col AND ...
```

```
SELECT col-1, col-2, ....  
FROM table-1 [INNER | EQUI | LEFT | RIGHT]  
JOIN table-2 ON table-1.Join_Col = table-2. Join_Col  
JOIN table-3 ON table-1.Join_Col = table-3. Join_Col  
[WHERE clause]
```

Joining More Than Two Tables

```
SELECT NAME, C. CUSTID, O. ORDID, ITEMID  
FROM CUSTOMER C, ORD O, ITEM I  
WHERE C. CUSTID = C. CUSTID AND  
      O. ORDID = I. ORDID
```

```
SELECT NAME, C. CUSTID, O. ORDID, ITEMID  
FROM CUSTOMER C, ORD O, ITEM I  
JOIN ORD O ON C. CUSTID = C. CUSTID  
JOIN ITEM I ON O. ORDID = I. ORDID
```

Joining More Than Two Tables

```
SELECT NAME, C. CUSTID, O. ORDID, ITEMID  
FROM CUSTOMER C, ORD O, ITEM I  
WHERE C. CUSTID = C. CUSTID AND  
      O. ORDID = I. ORDID
```

```
SELECT E.ENAME, D.DEPTNO, DNAME, GRADE  
FROM EMP E  
JOIN DEPT D ON E. DEPTNO = D.DEPTNO  
JOIN SALGRADE ON SAL BETWEEN LOSAL AND HISAL
```

Practice Questions

1. FIND THOSE EMPLOYEES WHO ARE WORKING IN ACCOUNTING DEPARTMENT.
2. FIND THOSE EMPLOYEES WHOSE DEPARTMENT LOCATION IS NEWYORK.
3. Find the employees who have not been assigned to any department. (Rovers)
4. LIST THOSE DEPARTMENTS WHO DOES NOT HAVE ANY EMPLOYEES.
5. FIND THE MANAGER OF 'SCOTT'.



Thank you for your attention.

Talia Noreen

Lecturer

Beaconhouse National University

School of Computer and Information Technology (SCIT)

13 Km Niazbeg Raiwind Road off Thokar, Beacon House Society

Lahore, Pakistan

E-mail: talia.noreen@bnu.edu.pk